Enterprise Multimodal Hybrid RAG System

Privacy-Preserving Local Document Intelligence with GPU Acceleration & Reciprocal Rank Fusion

Author / Presenter: AI Engineering Team	**Target Architecture:** Local Self-Hosted Stack
Core Model: Mistral 7B Instruct v0.2 (GGUF CUDA)	**Database:** PostgreSQL 16 + pgvector
Hardware Acceleration: NVIDIA RTX 3050 GPU	**Status:** Fully Production-Ready

1. Problem Statement & Core Objectives

■■ The Challenges in Traditional RAG

- **Privacy & Data Leaks:** Sending sensitive PDFs to external cloud LLM APIs (OpenAI/Anthropic) violates compliance.
- **High Latency & UI Freezes:** Heavy non-streaming queries cause 20+ second freezes on client applications.
- **Single-Vector Keyword Misses:** Pure dense embeddings often miss exact alphanumeric codes (e.g. course IDs like CS4674).
- **Multimodal Loss:** Diagrams, flowcharts, and structured tables inside PDFs are discarded.

■ Our Solution & System Objectives

- **100% Local Privacy:** Runs fully self-hosted without external API calls or network leaks.
- **GPU Acceleration:** Offloads LLM layers to NVIDIA RTX 3050 GPU for <1.5s streaming response times.
- **Hybrid Retrieval (RRF):** Combines BM25 lexical match + Dense vector similarity.
- **Multimodal Ingestion:** Extracts text, multi-column tables, and image captions into PostgreSQL.

2. System Architecture & End-to-End Pipeline

STAGE 1: INGESTION PIPELINE

PDF Document → PyMuPDF Extractor → Text/Image Chunker → Nomic 768d Embeddings → PostgreSQL (pgvector)

STAGE 2: HYBRID RETRIEVAL & RERANKING

User Query → [BM25 Lexical Match + Dense Cosine Vector Match] → Reciprocal Rank Fusion (RRF) → Cross-Encoder Reranker

STAGE 3: GPU LLM INFERENCE & REAL-TIME STREAMING

Reranked Passages + Context Prompt → Mistral-7B GGUF (NVIDIA RTX 3050 GPU) → Server-Sent Events (SSE) → Web UI

3. Technical Stack & Architectural Design

Component Layer	Technology Implemented	Engineering Rationale
Database	PostgreSQL 16 + pgvector	Unified relational + high-dimensional 768d vector storage.
Dense Embedder	Nomic-Embed-Text-v1.5	High performance 768d semantic representation.
Sparse Search	BM25 Okapi Algorithm	Ensures exact keyword matching for codes & technical terms.
Reranker	ms-marco-MiniLM-L-6-v2	Cross-Encoder re-scoring eliminates retrieval false positives.
LLM Inference	Mistral 7B Instruct (CUDA)	Offloads all 35 layers to 6GB RTX 3050 GPU VRAM.
Web Server & UI	FastAPI + SSE + Glassmorphic HTML	Non-blocking async streaming UI with live latency feedback.

4. Concrete Retrieval & Query Demonstrations

Case Study 1: Structured Curriculum Lookup	Case Study 2: Multimodal Architecture Query
Input Query: *'What is the course code for Information Retrieval and how many credits is it?'* **Retrieval Execution:** • BM25 matches code string CS4674. • Reranker score: 0.982 for Chunk #14. **Generated LLM Output:** *'The course code for Information Retrieval is CS4674. It is categorized as a Professional Elective with 3 credit hours.'* ■ **Latency: 0.85s (GPU Accelerated)**	**Input Query:** *'What does the architecture diagram on page 2 illustrate?'* **Retrieval Execution:** • Matches extracted image caption vector. • Retrieves page 2 image chunk + adjacent text. **Generated LLM Output:** *'The diagram on page 2 illustrates the system architecture, showing MediaPipe landmark tracking fed into OpenCV and Transformer classifiers.'* ■ **Latency: 1.12s (GPU Accelerated)**

5. Performance Benchmarks & Optimizations

LLM Model Offload	0 Layers (CPU RAM)	35 / 35 Layers (GPU VRAM)	100% GPU Offload
Token Generation Latency	25.86 seconds	1.20 seconds	**20.5x Faster**
Retrieval Pipeline Speed	0.65 seconds	0.56 seconds	Instant Match
Model Load Delay on Query	16.0s (Lazy-loaded)	0.0s (Pre-warmed on Boot)	**Zero Delay**

Key Engineering Optimizations Implemented:

1. **FastAPI Server Warmup:** Pre-loads embedding, reranker, and LLM models on server startup so initial query latency is instant.
2. **Async Event Loop:** Background document ingestion via subprocess prevents thread blocking during PDF processing.
3. **Dynamic BM25 Invalidation:** Checks database row count to auto-rebuild lexical index upon new PDF uploads.

6. Project Codebase Architecture & File Summary

Module Path	Key Responsibilities
app/api/server.py	FastAPI server hosting /upload, /ingest, /query/stream endpoints + startup model preloader.
app/api/static/index.html	Glassmorphic frontend UI consuming SSE token stream and rendering instant latency timer badges.
app/llm/local_llm.py	Local LLM wrapper managing llama-cpp-python with CUDA DLL path registration & GPU layer offloading.
app/retrieval/hybrid_retriever.py	Reciprocal Rank Fusion (RRF) algorithm combining BM25 lexical scores with dense vector similarity.
app/retrieval/reranker.py	Cross-Encoder transformer re-scoring top retrieved passages.
app/vectorstore/pgvector_store.py	PostgreSQL 16 connector managing auto-table schema creation and 768d vector queries.

Summary for Evaluation Committee: The project successfully achieves a 100% local, privacy-preserving, production-grade Multimodal RAG system running at lightning speeds (<1.5s) on an NVIDIA RTX 3050 GPU.